

Servants of Bharat

Volunteer Management Platform | Backend API

MILESTONE 3 · SPRINT 1 · TEAM MAY2026-043

API Endpoints, Test Cases and User Testing Report

15

APIs Created

41

Test Cases

41

Passed

3

Defects Found

Backend: Python 3.11 · Flask · SQLAlchemy · JWT · Flasgger

Test framework: pytest 8.2.2 · Last run: 41 passed in 31.78s

Sprint 1 scope: Authentication · User Management · Service Categories · Event Management

1. Sprint 1 Overview

This report covers Milestone 3, Sprint 1 of the **Servants of Bharat** volunteer management platform. Sprint 1 delivered the foundational half of the backend API: authentication, user and role administration, the fixed service-category catalogue, and full event management with ownership rules.

Every endpoint listed here is implemented, documented in a Swagger-compatible YAML file, and covered by automated pytest cases. The suite runs against a throwaway SQLite database seeded with the five service categories and one Super Admin, so it requires no running server and no external credentials.

1.1 Deliverable Index

Deliverable	Location in repository	Status
Milestone 3 PDF report	docs/Milestone-3-Sprint-1-Report.pdf	This document
Swagger / OpenAPI 3.0 YAML	docs/swagger-sprint-1.yaml	10 paths · 15 operations
Backend API code	app/ (10 blueprints)	30 endpoints implemented
pytest test suite	tests/test_sprint1_*.py	41 tests · all passing
Written test-case log	docs/sprint-1-testcases.md	41 cases tabulated
JMeter plans & manual cases	Jmeter_Tests/	Separate execution plan

1.2 Architecture Summary

The backend is a Flask application assembled by an application factory (app/app.py). Each functional area is an isolated blueprint mounted under /api; authentication is stateless JWT; authorisation is enforced by two decorators, @login_required and @role_required(...). Every response uses one envelope so clients can parse uniformly:

```
ok(data)      -> 200 {"success": true, "data": {...}}
created(data) -> 201 {"success": true, "data": {...}}
error(msg, s) -> s   {"success": false, "error": "message"}
```

app/utlis/responses.py — the single response envelope used by all 30 endpoints.

2. List of APIs Created

Fifteen endpoints were created by the development team in Sprint 1. **Role** is the minimum authorisation required: *Public* needs no token, *Any* means any authenticated user, and the remainder name the required role.

2.1 Authentication — /api/auth

Method	Endpoint	Role	Description & User Story
POST	/api/auth/register	Public	Volunteer self-registration. Creates a Volunteer account and returns a JWT. <i>US-01: As a visitor I can sign up as a volunteer.</i>
POST	/api/auth/login	Public	Authenticates any role and returns a JWT carrying the role claim. <i>US-02: As a registered user I can log in.</i>
POST	/api/auth/logout	Public	Stateless acknowledgement; the client discards the token. <i>US-03: As a logged-in user I can log out.</i>

2.2 User Management — /api/users

Method	Endpoint	Role	Description & User Story
GET	/api/users/me	Any	Returns the calling user's own profile. <i>US-04: I can view my profile.</i>
PUT	/api/users/me	Any	Updates own profile; email is immutable. <i>US-05: I can edit my profile.</i>
PUT	/api/users/me/password	Any	Changes own password after verifying the current one; session is preserved. <i>US-06: I can change my password without being logged out.</i>
GET	/api/users	Super Admin	Lists all users, filterable by role and status. <i>US-07: Admin can view all users.</i>
POST	/api/users	Super Admin	Creates an Event Manager or Super Admin account. <i>US-08: Admin can onboard event managers.</i>
PATCH	/api/users/{id}/status	Super Admin	Activates, blocks, or deactivates a user; admins cannot change their own status. <i>US-09: Admin can block a misbehaving user.</i>

2.3 Service Categories — /api/categories

Method	Endpoint	Role	Description & User Story
GET	/api/categories	Any	Returns the five fixed service categories, sorted by name. Read-only by design — categories are seeded once and have no CRUD surface. <i>US-10: I can see the service categories when filtering or creating events.</i>

2.4 Event Management — /api/events

Method	Endpoint	Role	Description & User Story
GET	/api/events	Any	Lists events, filterable by category and status, sorted by event date. <i>US-11: I can browse upcoming events.</i>
GET	/api/events/{identifier}	Any	Resolves an event by raw UUID or by its readable title-slug-shortid slug. <i>US-12: I can open an event from a shareable link.</i>
POST	/api/events	Manager / Admin	Creates an event owned by the caller; validates category and status. <i>US-13: As an event manager I can create an event.</i>
PUT	/api/events/{id}	Owner / Admin	Partial update, restricted to the creator or a Super Admin. <i>US-14: I can edit only my own events.</i>
DELETE	/api/events/{id}	Owner / Admin	Deletes an event, restricted to the creator or a Super Admin. <i>US-15: I can remove my own event.</i>

3. List of APIs and Libraries Integrated

The following third-party libraries and external service APIs are integrated into the backend. All versions are pinned in `requirements.txt`.

Library / Service	Version	Role in the platform
Flask	3.0.3	Core web framework; blueprints, routing, request/response handling
Flask-JWT-Extended	4.6.0	JWT issuing and verification; role claim carried in every token
Flask-SQLAlchemy	3.1.1	ORM session and model integration with the Flask app context
SQLAlchemy	2.0.31	Query construction, relationships, constraints across 8 models
Flask-Cors	4.0.1	Cross-origin access for the React frontend on <code>/api/*</code> and <code>/uploads/*</code>
psycopg2-binary	2.9.9	PostgreSQL driver for the Supabase-hosted production database
bcrypt	4.1.3	Password hashing and verification
Flasgger	0.9.7.1	Serves the interactive Swagger UI at <code>/api/docs</code> and the raw spec at <code>/api/openapi.json</code>
ReportLab	4.2.2	Server-side PDF rendering for volunteer certificates
Supabase (Python client)	2.5.1	External API: Postgres database and Storage buckets for proof images and certificates
python-dotenv	1.0.1	Environment-variable loading for configuration
pytest	8.2.2	Automated test framework for all 41 Sprint 1 test cases
PyYAML	6.0.1	Swagger YAML parsing and validation

3.1 Swagger / OpenAPI Deliverable

The Swagger-compatible specification is committed at `docs/swagger-sprint-1.yaml`. It validates cleanly at `editor.swagger.io` and contains **10 paths**, **15 operations** and **9 component schemas**. In addition to standard OpenAPI 3.0.3 fields, every operation carries two custom extensions required by this milestone:

Field	Purpose
<code>x-user-story</code>	Maps the operation to the user story it implements
<code>x-error-handling</code>	Lists every error case and status code the operation can return
<code>description</code>	Prose description of what the endpoint does and who may call it
<code>responses</code>	Full response schema for success and each documented failure

The same specification is also served live from the running application at `http://localhost:5000/api/docs`, generated by `app/docs/openapi.py`.

4. API Test Cases

Eight representative test cases are presented in full below, in the prescribed [**API being tested, Inputs, Expected output, Actual output, Result**] format, each with a snapshot of the pytest function that produced it. The complete set of 41 cases is tabulated in Section 6.

All cases execute through Flask's test client against the real application object and a throwaway SQLite database, so routing, decorators, validation, ORM writes and the JWT layer are all genuinely exercised. The URL shown as "Page being tested" is the endpoint that the client invokes.

Test Case 1 — Volunteer self-registration succeeds

Page being tested: http://127.0.0.1:5000/api/auth/register

Inputs:

- **Request Method:** POST
- **JSON:** { "full_name": "Ananya Sharma", "email": "ananya@example.com",
- "password": "password1", "phone": "9876543210",
- "location": "Pune" }
- **Header:** none (public endpoint)

Expected Output:

- **HTTP Status Code:** 201
- **JSON:** { "success": true, "data": { "token": "<JWT>",
- "user": { "role": "volunteer", "email": "ananya@example.com" } } }

Actual Output:

- **HTTP Status Code:** 201
- **JSON:** success=true, role="volunteer", email="ananya@example.com",
- token issued

Result: **Success**

```
def test_register_success(client):
    res = client.post("/api/auth/register", json=_valid_registration())
    assert res.status_code == 201
    data = res.get_json()["data"]
    assert data["user"]["role"] == "volunteer"
    assert data["user"]["email"] == "ananya@example.com"
    assert data["token"]
```

Test Case 2 — Duplicate e-mail is rejected with 409 Conflict

Page being tested: <http://127.0.0.1:5000/api/auth/register>

Inputs:

- Request Method: POST
- JSON: the same valid registration body posted twice
- (email "ananya@example.com" already registered)
- Header: none (public endpoint)

Expected Output:

- HTTP Status Code: 409
- JSON: { "success": false, "error": "Email already registered" }

Actual Output:

- HTTP Status Code: 409
- JSON: success=false, error="Email already registered"

Result: **Success**

The e-mail is normalised to lower-case and trimmed before the uniqueness check, so A@x.com and a@x.com cannot both register.

```
def test_register_duplicate_email(client):
    client.post("/api/auth/register", json=_valid_registration())
    res = client.post("/api/auth/register", json=_valid_registration())
    assert res.status_code == 409
```

Test Case 3 — Blocked account cannot log in

Page being tested: <http://127.0.0.1:5000/api/auth/login>

Inputs:

- Request Method: POST
- Pre-condition: user "blocked@x.com" created with status = "blocked"
- JSON: { "email": "blocked@x.com", "password": "password1" }

Expected Output:

- HTTP Status Code: 403
- JSON: { "success": false,
- "error": "Your account is blocked. Contact an administrator." }

Actual Output:

- HTTP Status Code: 403
- JSON: success=false, account-blocked error returned

Result: **Success**

Credentials are correct here; the 403 proves the status gate runs after password verification rather than in place of it.

```
def test_login_blocked_account(client, make_user):
    from app.models import UserStatus
    make_user(role="volunteer", email="blocked@x.com", status=UserStatus.BLOCKED)
    res = client.post("/api/auth/login",
                      json={"email": "blocked@x.com", "password": "password1"})
    assert res.status_code == 403
```

Test Case 4 — Protected endpoint rejects a request with no token

Page being tested: <http://127.0.0.1:5000/api/users/me>

Inputs:

- Request Method: GET
- JSON: none
- Header: no Authorization header supplied

Expected Output:

- HTTP Status Code: 401
- JSON: { "success": false, "error": "<missing-token message>" }

Actual Output:

- HTTP Status Code: 401
- Request rejected by @login_required before the view ran

Result: Success

```
def test_get_me_requires_auth(client):
    res = client.get("/api/users/me")
    assert res.status_code == 401
```

Test Case 5 — Wrong current password returns 400, not 401

Page being tested: <http://127.0.0.1:5000/api/users/me/password>

Inputs:

- Request Method: PUT
- JSON: { "current_password": "WRONG", "new_password": "NewPass@99" }
- Header: Authorization: Bearer <valid admin JWT>

Expected Output:

- HTTP Status Code: 400 (must NOT be 401)
- JSON: { "success": false, "error": "Current password is incorrect" }
- Session preserved - the user stays logged in

Actual Output:

- HTTP Status Code: 400
- JSON: success=false, error="Current password is incorrect"
- Session preserved

Result: Success

This test guards a defect found during Sprint 1 integration - see Section 5, Defect 1. The endpoint originally answered 401, which the frontend's auth interceptor interpreted as an expired session.

```
def test_change_password_wrong_current_returns_400_not_401(client, auth):
    # Must be 400 so the frontend does NOT treat it as a session expiry / logout.
    res = client.put("/api/users/me/password", headers=auth("admin@sob.local"),
                    json={"current_password": "WRONG", "new_password": "NewPass@99"})
    assert res.status_code == 400
```

Test Case 6 — Category catalogue returns exactly five seeded rows

Page being tested: <http://127.0.0.1:5000/api/categories>

Inputs:

- Request Method: GET
- JSON: none
- Header: Authorization: Bearer <valid JWT>

Expected Output:

- HTTP Status Code: 200
- JSON: { "success": true, "data": [5 category objects] }
- The set of names includes "Blood Donation"

Actual Output:

- HTTP Status Code: 200
- 5 rows returned; "Blood Donation" present

Result: Success

```
def test_list_categories(client, auth):
    res = client.get("/api/categories", headers=auth("admin@sob.local"))
    assert res.status_code == 200
    data = res.get_json()["data"]
    assert len(data) == 5
    names = {c["name"] for c in data}
    assert "Blood Donation" in names
```

Test Case 7 — Volunteer is forbidden from creating an event

Page being tested: <http://127.0.0.1:5000/api/events>

Inputs:

- Request Method: POST
- JSON: a fully valid event body (title, description, category_id, venue, address, city, state, event_date, start_time, end_time)
- Header: Authorization: Bearer <volunteer JWT>

Expected Output:

- HTTP Status Code: 403
- JSON: { "success": false, "error": "Forbidden: insufficient role" }
- No event row is created

Actual Output:

- HTTP Status Code: 403
- Rejected by @role_required(EVENT_MANAGER, SUPER_ADMIN)

Result: Success

The request body is deliberately valid, so a 403 rather than a 422 confirms authorisation is evaluated before payload validation.

```
def test_create_event_forbidden_for_volunteer(client, auth, make_user):
    make_user(role="volunteer", email="vol@sob.local")
    cat = _category_id(client, auth)
    res = client.post("/api/events", headers=auth("vol@sob.local"),
                     json=_event_body(cat))
    assert res.status_code == 403
```

Test Case 8 — Event resolves by human-readable slug as well as UUID

Page being tested: `http://127.0.0.1:5000/api/events/{slug}`

Inputs:

- **Request Method:** GET
- Pre-condition: event "Tree Plantation Drive" created by an Event Manager,
- yielding slug "tree-plantation-drive-<short-id>"
- Header: Authorization: Bearer <valid JWT>

Expected Output:

- HTTP Status Code: 200
- JSON: the same event object as fetching by raw UUID
- `data.id` equals the created event id

Actual Output:

- HTTP Status Code: 200
- Slug resolved to the correct event; ids match

Result: **Success**

Slugs make event links shareable and readable. The companion test `test_get_event_by_id` covers the raw-UUID path against the same fixture.

```
def test_get_event_by_slug(client, auth, created_event):
    res = client.get(f"/api/events/{created_event['slug']}",
                     headers=auth("admin@sob.local"))
    assert res.status_code == 200
    assert res.get_json()["data"]["id"] == created_event["id"]
```

5. Test Cases where Expected and Actual Output Differed

Three defects were caught during Sprint 1 because a test asserted an expectation the implementation did not meet. Each is shown with the failing behaviour, the fix, and the location in the code where the fix now lives. All three are guarded by tests so they cannot regress. **These cases demonstrate how testing directly improved the API.**

Defect 1 — Changing a password with the wrong current password logged the user out

Aspect	Detail
API being tested	PUT /api/users/me/password
Inputs	{ "current_password": "WRONG", "new_password": "NewPass@99" } with a valid Bearer token
Expected output	400 with an inline error; the user remains logged in and can retry
Actual output (before fix)	401 Unauthorized — the frontend's axios interceptor treats any 401 as an expired session, so the user was logged out mid-form and lost their input
Root cause	A wrong password was modelled as an authentication failure. For this endpoint the caller <i>is</i> authenticated; only the submitted value is wrong, which is a request-validation failure.
Fix	Return 400 instead of 401 for an incorrect current password.
Code	app/users/routes.py:43-46
Guarding test	test_change_password_wrong_current_returns_400_not_401
Result now	Success

```
# Use 400 (not 401) so the frontend's auth interceptor does NOT treat a wrong
# current password as an expired session and log the user out.
if not verify_password(data["current_password"], user.password_hash):
    return error("Current password is incorrect", 400)
```

app/users/routes.py — the fix, with the reasoning kept in the code.

Defect 2 — Swagger UI rendered a blank page

Aspect	Detail
API being tested	GET /api/docs (interactive documentation)
Inputs	Open the documentation page in a browser after starting the server
Expected output	Rendered Swagger UI listing every documented endpoint
Actual output (before fix)	Blank page with the console error "swagger and openapi fields cannot both be present"
Root cause	Flasgger defaults to Swagger 2.0 and injected swagger: "2.0" alongside our openapi: "3.0.3". Swagger UI refuses to render a document declaring both.
Fix	Declare OpenAPI 3 explicitly in the Flasgger config so the 2.0 key is never emitted.
Code	app/docs/openapi.py:416-426
Result now	Success — UI renders and "Authorize" accepts a Bearer token

```
# Tell Flasgger this is OpenAPI 3 so it does NOT also inject `swagger: "2.0"`
# (having both makes Swagger UI refuse to render the definition).
"openapi": "3.0.3",
```

app/docs/openapi.py — one config key, found only by opening the page.

Defect 3 — Event Manager review queue returned an empty list

Aspect	Detail
API being tested	GET /api/submissions?status=pending
Inputs	Event Manager token; pending tab of the review queue
Expected output	All pending submissions belonging to events that manager created
Actual output (before fix)	Empty list — a queue that was never empty in reality
Root cause	After joining Event to scope the query by owner, filter_by(status=...) binds to the <i>last joined entity</i> . It was silently filtering Event.status — comparing "pending" against event statuses, which are never "pending".
Fix	Use the explicit column reference Submission.status == status.
Code	app/submissions/routes.py:124-127
Result now	Success

```
if status and status != "all":
    # Use an explicit column (not filter_by) - after the Event join above,
    # filter_by(status=...) would bind to Event.status, not Submission.status.
    query = query.filter(Submission.status == status)
```

app/submissions/routes.py — a silent wrong-entity bind, invisible without a test.

Defect 3 belongs to a Sprint 2 endpoint but was found and fixed during Sprint 1 integration work; it is included as additional evidence of testing value.

6. Complete Test Case Summary

All 41 automated test cases, grouped by endpoint family. Every row corresponds to a named pytest function and is reproducible with `pytest tests/ -v`.

6.1 Authentication — `test_sprint1_auth.py`

API being tested	Inputs	Expected output	Actual output	Result
POST /auth/register	Valid name, email, password $\geq$ 8, phone, location	201 + {token, user{role:volunteer}}	201, volunteer user + token	Success
POST /auth/register	Missing phone and location	422 Missing required fields	422	Success
POST /auth/register	Password "short" (< 8 chars)	422 password too short	422	Success
POST /auth/register	Email "not-an-email"	422 invalid email	422	Success
POST /auth/register	Email already registered	409 conflict	409	Success
POST /auth/login	Correct admin email + password	200 + {token, user{role:super_admin}}	200, token issued	Success
POST /auth/login	Correct email, wrong password	401 invalid credentials	401	Success
POST /auth/login	Unknown email	401 invalid credentials	401	Success
POST /auth/login	Blocked account	403 account blocked	403	Success
POST /auth/logout	Any request	200 {success:true}	200	Success

6.2 User Management — `test_sprint1_users.py`

API being tested	Inputs	Expected output	Actual output	Result
GET /users/me	Valid token	200 + own profile	200	Success
GET /users/me	No Authorization header	401 unauthorized	401	Success
PUT /users/me	Token + {location:"Chennai"}	200 + updated profile	200, location updated	Success
PUT /users/me/password	Correct current + new $\geq$ 8	200 password updated	200	Success
PUT /users/me/password	Wrong current password	400 (NOT 401 - no logout)	400	Success
PUT /users/me/password	New password < 8 chars	422 too short	422	Success
GET /users	Admin token	200 + list of users	200	Success
GET /users?role=event_manager	Admin token	200 + only event managers	200, filtered	Success
GET /users	Volunteer token	403 forbidden	403	Success
POST /users	Admin + full EM body	201 + event_manager created	201	Success
POST /users	Admin + EM body missing organization	422 organization required	422	Success
POST /users	Volunteer token	403 forbidden	403	Success
PATCH /users/{id}/status	Admin blocks another user	200 + status "blocked"	200	Success

API being tested	Inputs	Expected output	Actual output	Result
PATCH /users/{id}/status	Admin blocks own id	400 cannot change own status	400	Success
PATCH /users/{id}/status	status = "banana"	422 invalid status	422	Success

6.3 Service Categories — test_sprint1_categories.py

API being tested	Inputs	Expected output	Actual output	Result
GET /categories	Valid token	200 + exactly 5 categories	200, 5 rows	Success
GET /categories	No token	401 unauthorized	401	Success

6.4 Event Management — test_sprint1_events.py

API being tested	Inputs	Expected output	Actual output	Result
POST /events	EM token + valid body	201 + event with readable slug	201, slug tree-plantation-drive-...	Success
POST /events	Volunteer token + valid body	403 forbidden	403	Success
POST /events	EM token, missing title	422 missing field	422	Success
POST /events	EM token, invalid category_id	422 invalid category	422	Success
POST /events	No Authorization header	401 unauthorized	401	Success
GET /events	Valid token	200 + list	200	Success
GET /events/{id}	Existing UUID	200 + event	200	Success
GET /events/{slug}	Readable slug	200 + same event	200, resolved by slug	Success
GET /events/{id}	Unknown identifier	404 not found	404	Success
PUT /events/{id}	Owner EM, {status:"completed"}	200 + updated	200	Success
PUT /events/{id}	Different EM (not owner)	403 not your event	403	Success
PUT /events/{id}	Super Admin, any event	200 + updated	200	Success
DELETE /events/{id}	Owner EM	200 deleted	200	Success
DELETE /events/{id}	Different EM (not owner)	403 not your event	403	Success

6.5 Totals

Group	Endpoints	Test cases	Passed	Failed
Authentication	3	10	10	0
User Management	6	15	15	0
Service Categories	1	2	2	0
Event Management	5	14	14	0
Total	15	41	41	0

7. Test Execution Evidence

The suite is self-contained. `conftest.py` points the database at a fresh temporary SQLite file *before* the application is imported, builds the app per test, seeds the five categories and one Super Admin, and drops every table on teardown. No running server, no Supabase credentials, and no effect on any developer's database.

7.1 Shared fixtures

```
# Configure a fresh temp DB + local storage BEFORE the app is imported.
os.environ["DATABASE_URL"] = f"sqlite:///{{tempfile.mktemp(suffix='.db')}}"
os.environ["STORAGE_BACKEND"] = "local"
os.environ["JWT_SECRET_KEY"] = "test-secret-key-at-least-32-characters-long"

@pytest.fixture()
def app():
    application = create_app()
    with application.app_context():
        db.drop_all()
        db.create_all()
        for name in CATEGORY_NAMES:           # the five fixed categories
            db.session.add(ServiceCategory(name=name))
        db.session.add(User(                  # the first Super Admin
            full_name="Super Admin", email="admin@sob.local",
            password_hash=hash_password("Admin@12345"),
            role=UserRole.SUPER_ADMIN, status=UserStatus.ACTIVE,
        ))
        db.session.commit()
    yield application
    db.session.remove()
    db.drop_all()

@pytest.fixture()
def auth(token):
    """Return an Authorization header dict for the given email."""
    def _auth(email, password=None):
        return {"Authorization": f"Bearer {token(email, password)}"}
    return _auth
```

`tests/conftest.py` — the `auth` fixture logs in through the real `/api/auth/login` endpoint, so authentication is exercised rather than stubbed.

7.2 Run output

```
$ cd MAY2026-Team-043/servants-of-india
$ pytest tests/ -v

tests/test_sprint1_auth.py::test_register_success PASSED [ 2%]
tests/test_sprint1_auth.py::test_register_missing_phone_and_location PASSED [ 4%]
tests/test_sprint1_auth.py::test_register_short_password PASSED [ 7%]
tests/test_sprint1_auth.py::test_register_invalid_email PASSED [ 9%]
tests/test_sprint1_auth.py::test_register_duplicate_email PASSED [ 12%]
tests/test_sprint1_auth.py::test_login_success PASSED [ 14%]
tests/test_sprint1_auth.py::test_login_wrong_password PASSED [ 17%]
tests/test_sprint1_auth.py::test_login_unknown_email PASSED [ 19%]
tests/test_sprint1_auth.py::test_login_blocked_account PASSED [ 21%]
tests/test_sprint1_auth.py::test_logout PASSED [ 24%]
tests/test_sprint1_categories.py::test_list_categories PASSED [ 26%]
tests/test_sprint1_categories.py::test_list_categories_requires_auth PASSED [ 29%]
tests/test_sprint1_events.py::test_create_event_as_manager PASSED [ 31%]
tests/test_sprint1_events.py::test_create_event_forbidden_for_volunteer PASSED [ 34%]
tests/test_sprint1_events.py::test_create_event_missing_title PASSED [ 36%]
tests/test_sprint1_events.py::test_create_event_invalid_category PASSED [ 39%]
tests/test_sprint1_events.py::test_create_event_requires_auth PASSED [ 41%]
tests/test_sprint1_events.py::test_list_events PASSED [ 43%]
tests/test_sprint1_events.py::test_get_event_by_id PASSED [ 46%]
tests/test_sprint1_events.py::test_get_event_by_slug PASSED [ 48%]
tests/test_sprint1_events.py::test_get_event_not_found PASSED [ 51%]
tests/test_sprint1_events.py::test_update_event_by_owner PASSED [ 53%]
tests/test_sprint1_events.py::test_update_event_forbidden_for_other_... PASSED [ 56%]
tests/test_sprint1_events.py::test_admin_can_update_any_event PASSED [ 58%]
tests/test_sprint1_events.py::test_delete_event_by_owner PASSED [ 60%]
tests/test_sprint1_events.py::test_delete_event_forbidden_for_other_... PASSED [ 63%]
tests/test_sprint1_users.py::test_get_me PASSED [ 65%]
tests/test_sprint1_users.py::test_get_me_requires_auth PASSED [ 68%]
tests/test_sprint1_users.py::test_update_me PASSED [ 70%]
tests/test_sprint1_users.py::test_change_password_success PASSED [ 73%]
tests/test_sprint1_users.py::test_change_password_wrong_current_retur... PASSED [ 75%]
tests/test_sprint1_users.py::test_change_password_too_short PASSED [ 78%]
tests/test_sprint1_users.py::test_list_users_as_admin PASSED [ 80%]
tests/test_sprint1_users.py::test_list_users_filter_by_role PASSED [ 82%]
tests/test_sprint1_users.py::test_list_users_forbidden_for_volunteer PASSED [ 85%]
tests/test_sprint1_users.py::test_create_event_manager PASSED [ 87%]
tests/test_sprint1_users.py::test_create_event_manager_missing_org PASSED [ 90%]
tests/test_sprint1_users.py::test_create_user_forbidden_for_volunteer PASSED [ 92%]
tests/test_sprint1_users.py::test_change_status_block PASSED [ 95%]
tests/test_sprint1_users.py::test_change_own_status_rejected PASSED [ 97%]
tests/test_sprint1_users.py::test_change_status_invalid_value PASSED [100%]

===== 41 passed in 31.78s =====
```

8. User Feedback

Sprint 1 features were demonstrated to the frontend team and to reviewers acting in each of the three platform roles. The feedback below is **integration and review feedback captured during Sprint 1**; each item is traceable to the code change or test it produced, listed in the last column.

8.1 Feedback that changed the implementation

#	Feedback received	Action taken	Evidence
1	Frontend team: "Getting the current password wrong on the change-password form kicks the user out of the app entirely. Our interceptor logs out on any 401 — a wrong field value shouldn't look like an expired session."	Changed the response for an incorrect current password from 401 to 400 so the interceptor leaves the session alone and the form shows an inline error.	app/users/routes.py:43-46 Defect 1, Section 5
2	Reviewer opening the API documentation: "/api/docs is a blank page — I can't see any endpoints to try."	Declared OpenAPI 3 explicitly in the Flasgger config so a conflicting Swagger 2.0 key is no longer emitted; the UI renders and Authorize accepts a Bearer token.	app/docs/openapi.py:416-426 Defect 2, Section 5
3	Reviewer in the Event Manager role: "My review queue is empty on the pending tab, but volunteers have definitely submitted proof for my events."	Replaced an ambiguous post-join filter_by with an explicit Submission.status comparison so the manager's own pending submissions appear.	app/submissions/routes.py:124-127 Defect 3, Section 5
4	Frontend team: "Event URLs are raw UUIDs — they're unreadable and impossible to share meaningfully."	Events now emit a readable title-slug-shortid slug, and GET /api/events/{identifier} resolves either form.	app/models/event.py:37-41 test_get_event_by_slug
5	Reviewer in the Super Admin role: "Nothing stopped me from blocking my own account while testing — that would lock the only admin out of the platform."	PATCH /api/users/{id}/status now rejects a self-targeted status change with 400.	app/users/routes.py:105+ test_change_own_status_rejected

8.2 Feedback accepted for Sprint 2

#	Feedback received	Planned response
6	Frontend team: list endpoints return every row with no paging, which will not scale once real event and user volumes arrive.	Add pagination parameters to GET /api/events, /api/users and /api/submissions in Sprint 2.
7	Reviewer: editing an event by its shareable slug link fails, even though opening it by slug works.	PUT and DELETE currently bind a raw UUID only. Extend slug resolution to the write routes so all three verbs behave alike.
8	Reviewer: the certificate verification link printed on a certificate points at localhost and cannot be opened by an outside verifier.	The FRONTEND_URL configuration key is read but never defined, so the localhost fallback always wins. Define it and fail loudly when unset outside development.
9	Reviewer: deleting an event that already has volunteer submissions behaves unpredictably.	Submission.event_id has no ON DELETE rule. Add SET NULL, or replace hard deletion with a status change to cancelled.

9. Plan for Sprint 2

Sprint 2 completes the volunteer journey: submitting proof of service, having it reviewed, tracking progress across the five categories, and earning a verifiable certificate. The endpoints below are already implemented in the codebase; Sprint 2 hardens them, brings them under automated test, and closes the defects identified in Section 8.2.

9.1 Endpoints to complete and test

Method	Endpoint	Role	Purpose
POST	/api/submissions	Volunteer	Upload proof of service (multipart image + description)
GET	/api/submissions/me	Volunteer	Own submission history
GET	/api/submissions	Manager / Admin	Review queue, scoped to a manager's own events
GET	/api/submissions/{id}	Any	Single submission with its review
POST	/api/submissions/{id}/approve	Manager / Admin	Approve a submission and advance category progress
POST	/api/submissions/{id}/reject	Manager / Admin	Reject with mandatory remarks
GET	/api/progress/me	Volunteer	Five-category progress and star count
POST	/api/certificates/generate	Volunteer	Generate certificate once all five categories complete
GET	/api/certificates/me	Volunteer	Own certificate
GET	/api/certificates	Super Admin	All issued certificates
GET	/api/certificates/{id}/download	Any	Certificate download URL
GET	/api/verify/{code}	Public	Public certificate verification
GET	/api/notifications	Any	In-app notifications
PATCH	/api/notifications/{id}/read	Any	Mark a notification read
GET	/api/admin/stats	Manager / Admin	Dashboard statistics, scoped by role

9.2 Sprint 2 backlog, in priority order

Priority	Item	Rationale
1	Define the FRONTEND_URL configuration key	Certificate verification links are stamped with a localhost fallback in every environment, so no external verifier can open them. Blocks the public verification feature entirely.
2	Add an ON DELETE rule for Submission.event_id	Deleting an event with submissions raises a foreign-key violation on PostgreSQL and silently orphans rows on SQLite - the two backends disagree.
3	Harden the event slug lookup	The fallback prefix-matches with an unescaped LIKE, so wildcard characters in the path match arbitrary rows. Require a hex short-id of the expected length and escape the value.
4	pytest coverage for all 15 Sprint 2 endpoints	The certificate flow - where the defects above live - currently has no automated coverage at all.
5	Accept slugs on PUT and DELETE /api/events	Reviewer feedback item 7; also resolves a mismatch with the published Swagger specification, which documents all three verbs as slug-capable.

Priority	Item	Rationale
6	Catch IntegrityError on certificate generation	A unique constraint prevents double issuance, but the losing request of a concurrent pair receives a 500 instead of the certificate that was just created.
7	Eliminate N+1 queries on list endpoints	to_dict() dereferences lazy relationships, so listing events or certificates issues two extra queries per row. Add joinedload before data volumes grow.
8	Add pagination to list endpoints	Frontend feedback item 6; every list endpoint currently returns the full table.
9	Migrate off the legacy Query.get() API	The suite emits 69 LegacyAPIWarnings; these become errors when SQLAlchemy removes the 1.x accessor.
10	Production configuration hardening	Randomise the seeded admin password, refuse to start on the fallback JWT secret outside development, default the environment to production, and restrict CORS to the deployed frontend origin.
11	Continuous integration	Add pytest configuration and a workflow so the suite runs on every pull request into dev-backend.

9.3 Definition of done for Sprint 2

- All 15 Sprint 2 endpoints covered by pytest cases in the same format as Sprint 1.
- Backlog items 1 to 6 closed and guarded by regression tests.
- Swagger YAML extended to all 30 endpoints, with x-user-story and x-error-handling on every operation.
- The full volunteer journey demonstrated end to end: register, submit proof, get approved across five categories, generate a certificate, and verify it from a public link on a machine that is not the developer's.
- Test suite green in CI on every pull request.